

宿说—酒店评论智能顾问·技术报告

- 项目: 酒店评论跨模态检索与智能问答系统, 链接: <https://github.com/Leucanthos/All-about-the-Hotel>
- 数据: 6441 条广州花园酒店真实评论 • 1233 张酒店, 图片 • 4 种房型
- 成员: 苏俊儒 (多模态检索) • 孙宇豪 (Agentic RAG + SFT) • 董依心 (聚类生成摘要 + 增强检索算法)

一、项目概述

`hotel` 是一个终端命令。装好之后敲 `hotel` 进入交互模式, 输入出行场景或具体问题, 系统流式返回评估结果。背后是三个人各自独立开发的算法, 通过统一路由层自动调度。

两种典型的使用路径。**顾问咨询型**——“带 5 岁宝宝去住推荐吗”——系统先识别为亲子家庭场景, 再拆成安静程度、安全、儿童设施三个维度, 每个维度跑多路检索 (multimodal + rag), 最后交给 LLM 综合研判, 输出结构化回复。整个过程 **16 个事件**、约 **5 秒**。**事实查证型**——“有洗衣机吗”——命中关键词直接进 practical 捷径, 提取“洗衣机”做 BM25 检索, 去重展示, **6 个事件**、**0.3 秒**完成, 不调任何 API。

>>> 带5岁宝宝去住, 推荐吗?

正在分析你的出行场景...

识别为「亲子家庭」场景 (置信度 95%, 路由: llm)

拆解为 3 个核心维度

安静程度 → 「带宝宝出行需要安静环境, 房间隔音好不好?」

安全 → 「酒店安全措施怎么样? 适不适合带小孩?」

儿童设施 → 「有儿童乐园、亲子活动吗? 提供婴儿床吗?」

并发检索 3 个维度...

[1/3] 安静程度 -> 召回方式: multimodal, rag

[2/3] 安全 -> 召回方式: rag

[3/3] 儿童设施 -> 召回方式: rag

正在综合研判各维度结果...

综合结论: 谨慎推荐

(4.2s)

[结构化回复, 含各维度判断与住客原文引用]

安装入口: `python install.py` (一键装依赖、下载模型、构建索引、跑验证), 或者 `pip install -e .` 只装 CLI。

这门课的大作业。三个人从不同方向切入：苏俊儒搞多模态图文检索，孙宇豪搞 Agentic RAG 流水线加 SFT 微调，董依心搞 BM25、意图分类、RRF 融合等增强检索算法。一开始各写各的，到第三阶段要拼成一套系统的时候发现问题了：调用方式不统一（三个 API 入口各不一样）、数据各加载各的（同一个 parquet 文件被读了三遍）、没有路由层。用户不知道该用谁的算法搜什么东西。所以第三阶段的核心工作是整合：加统一路由做自动调度，再包 CLI 做交互入口。

数据是广州花园酒店携程评论，6441 条中文评论加 1233 张酒店实拍图片，覆盖 4 种房型。存在 data/hotel_reviews_full.parquet (84MB)。后期从 296MB CSV 扩展了 694 家酒店、414k 条评论用于多酒店测试，压缩为 Parquet 后 84MB，压缩比 3.4x。

系统分三层，但不是设计阶段规划好的，是开发中自然长出来的。先有三套独立算法 (Layer 1)，然后发现需要统一路由调度 (Layer 2)，最后包了 CLI 给用户用 (Layer 3)。核心设计原则就一条：用户不加参数时，整个链路跑在本地，不花 API 钱。

Layer 3 (应用)	CLI (hotel/) + 终端 Demo
Layer 2 (路由)	场景识别 → 两级分流 → 管线调度
Layer 1 (算法)	多模态CLIP BM25 RRF Agentic RAG SFT

二、算法设计

多模态图文检索（苏俊儒）

代码在 scripts/leuca/multimodal/engine.py (检索引擎)、api.py (统一 API)、build.py (索引构建)，以及 scripts/_shared/store.py (NumPy 向量存储)。

一开始用的是 openai/clip-vit-base-patch32。中文字符用英文 CLIP 的 BPE tokenizer 切出来 token 数是英文的 2-3 倍，检索精度明显差。换成 OFA-Sys/chinese-clip-vit-base-patch16 后中文文本编码质量好了很多。没考虑过更大的模型（如 Chinese-CLIP-Large），因为需要本地推理，large 在 CPU 上太慢。还探索过一次 Stable Diffusion 反向检索（文本生成图片再图片搜图片），生成的图片和真实酒店照片风格差太远，直接放弃了。

最初用 ChromaDB 管理向量，在 Windows 上遇到了一个诡异 bug：HNSW 索引的持久化写入随机失败，概率大约 10%，不报错不抛异常无法复现。花了两天的时间，换 HNSW 参数、换存储路径、换 ChromaDB 版本，都不行。最后决定放弃 ChromaDB，自己写基于 NumPy 的向量存储。_shared/store.py 核心逻辑不到 100 行：向量存 .npy，元数据存 JSON，余弦相似度用 np.dot 在内存算。单次查询 2-3ms，比 ChromaDB 还快一点（不需要跨进程通信）。代价是缺增量更新和分布式支持，但对 1200 张图片的规模完全够用。

微调实验跑了 6 组，全失败了。基于 960 条训练/240 条测试数据，对 Chinese-CLIP 的投影头做微调。详细记录在 scripts/leuca/__arch_research/docs/实验与优化.md。

#	方案	Test Hit@10	vs 基线
—	原始 Chinese-CLIP	5.4%	基准
E1	解冻顶层 1 层 Transformer	1.3%	↓
E2	数据增强 (RandomCrop + ColorJitter)	1.3%	↓
E3	可学习温度 + Label Smoothing 0.1	—	训练失败

#	方案	Test Hit@10	vs 基线
E4	max_length 77→200	1.3%	↓
E5	I2T loss 权重 3×	1.3%	↓
E6	难负样本加权	1.3%	↓

6 组微调全部失败。原因很直接：960 条数据太少，投影头微调对 P2I（文本搜图片）方向反而是损害。E3 直接训练失败，Label Smoothing 在小数据集上让 loss 震荡发散。

LTR 重排序做了 7 轮迭代（代码在 `scripts/leuca/__arch_research/ltr/`），在 CLIP top-20 召回结果上训练 MLP 重排序器。

Round	方法	Hit@1	Hit@5	洞察
R0	CLIP only	0%	1.0%	基线
R1	MLP 4feat pairwise	1.0%	7.0%	room_type 特征有效
R2	5feat listwise	3.0%	7.0%	Hit@1 最佳
R3	7feat deeper	2.0%	7.0%	天花板
R4	+ 跨模态特征	2.0%	7.0%	零增益
R5	+ 难负样本加权	2.0%	7.0%	零增益
R6	Siamese 细粒度	0%	1.0%	退化到基线以下

R1 有明显提升（Hit@5 从 1% 到 7%），但 R2 到 R6 全部停在 7%。分析了 CLIP 的 top-20 召回池后发现：**top-20 里只有 1.4% 是正确答案**，排序能操作的空间极小。R6 的 Siamese 模型可能因为过度拟合少量正样本，直接退化到随机水平。核心结论：**瓶颈在召回，不在排序**。CLIP 在多模态空间中并没有把酒店评论文本和酒店实拍图片对齐好——同一评论的文本-图片直接余弦相似度只有 **0.36**。这个根本问题不解决，排序怎么优化都没用。

还尝试了用 BERT 做房型分类器，先预测房型再在对应房型内检索。Frozen BERT + Linear 准确率 41%，Fine-tune 2 层加类别权重拉到 45%。**45% 就是天花板**——文本到房型的映射本身就是模糊的（“房间很舒服”到底是哪种房型？），人工标注的一致性都不到 60%。

推理加速方面，PyTorch CPU 上一次文本编码约 80ms。Intel Arc B390 用 OpenVINO 加速后降到 3.5ms（提速 23×），图片编码从 120ms 降到 3.5ms（提速 34×）。GPU 模型导出逻辑在 `engine.py` 的 `_init_openvino()`。

最终准确率（240 样本）：

指标	值	说明
Strict Hit@10	5.4%	精确命中目标图片（随机基线 0.8%）
Relaxed Hit@10（同房型）	51.5%	房型级别可用
Coverage	57%	超过一半图片曾被命中

错误来源：63% 语义鸿沟（“豪华”映射不到视觉特征）、33% 视觉混淆（不同房型照片相似）、4% 索引缺失。

Agentic RAG + SFT（孙宇豪）

代码在 `scripts/yuhao/agentic/engine.py`（HotelCorpus + AgenticRAG）、`api.py`（统一接口）、`scripts/_shared/llm.py`（LLM 调用抽象层）、`scripts/yuhao/sft/`（SFT 数据构建 + 微调）。

最开始的 RAG 是单步的：检索然后 LLM 生成。但复合查询有问题——“带老人小孩入住合适吗”，单步检索只能命中“老人”的评论，“小孩”的部分漏掉了。于是加了 Plan → Retrieve → Evaluate → Rewrite 的 Agent 循环（`engine.py` 的 `AgenticRAG.run()` 方法，约 80 行）。Plan 阶段分析查询关键词推断相关类别（“老人 + 小孩” → `safety + accessible + children_fac`），Retrieve 执行类别引导或全局检索，Evaluate 计算证据覆盖分（基于 BM25 分数加词重叠，低于 0.45 触发重试），Rewrite 重写查询追加类别提示后二次检索。

三组测试查询（“前台服务”、“早餐推荐”、“老人小孩”）都是一次检索就满足 `evidence score ≥ 1.0`。Agent 循环的重试分支实际上从未被触发过。但它保底——如果查询更刁钻（“酒店有没有适合三岁小孩和八十岁老人同时参加的活动”），这个循环就会起作用。

实际部署时发现 AgenticRAG 太慢了。一次完整循环（Plan+Retrieve+Evaluate）约 500ms，而大部分查询（“早餐怎么样”、“游泳池干净吗”）用 BM25 精确关键词匹配就够了，MRR 能到 100%。于是在 `_shared/retriever.py` 的 `_run_rag()` 里加了 **BM25-first 策略**（第 144-166 行）：先跑 BM25，如果 `top score ≥ 4.0` 且有 3 条以上结果就直接返回，跳过完整 RAG 循环。实测约 70% 的查询走这条捷径。

SFT 做了 1200 条训练数据（`scripts/yuhao/sft/build.py`），用 unsloth + QLoRA 4-bit 对 Qwen3-1.5B 微调（`scripts/yuhao/sft/train.py`），目标是让模型学会“先引用证据再回答”的格式，loss 降到 0.3 左右。部署时遇到问题：LoRA 权重合并回 base model 后，在 Intel Arc GPU 上用 OpenVINO 推理，延迟从基线的 2s 变成了 8s。根因是 QLoRA 的 4-bit 计算路径和 OpenVINO 的 INT8/FP16 优化路径不兼容，合并后的模型 fallback 到了未优化路径。最终决定暂时不部署 SFT，线上推理仍用 DeepSeek API。

RAG 管线还有一个容易被忽略的环节：类别摘要的非对称向量检索，由董依心实现并整合进 RAG 管线。人工标注了 14 个类别摘要（如“前台服务”、“餐饮设施”），每条约 650 字。但用户查询是短句（10-20 字），和 650 字长文之间的语义差距太大。解决方案是在查询和摘要之间插一层 QA 变体：为每个类别构造 3 条“假如已被回答”的假设句（约 50 字）。查询先搜 QA 变体（语义距离近），再通过文档关联字段桥接到完整摘要。检索参数：8 条候选池，余弦相似度阈值 $\tau=0.25$ 过滤，同类别去重只保留最高分。意图分类如果命中了某类别但向量检索没召回，用规则强制补充。召回的摘要以“类别背景知识”块注入 LLM prompt，位于具体评论之前，形成“宏观背景加具体证据”的双层结构。代码在 `scripts/dyx/engine.py` 的 `expand_query_llm()` 和反向 Query 检索逻辑。

RAG 召回指标：P@1 100%、P@5 96%、P@10 75%（最多返回 8 条 evidence）、MRR 100%。P@10 偏低是因为 RAG 定位是“结构化带证据的回答”而非纯文档召回。

聚类生成摘要 + 增强检索算法（董依心）

代码在 `scripts/_shared/bm25.py`（BM25 索引）、`router.py`（意图分类 + 路由）、`fusion.py`（RRF 融合）以及 `scripts/dyx/`（聚类、HyDE 实验）。开发过程中原始代码

散落在 `lib.py` 和 `lib_syh.py` 两个大文件里，第三阶段重构为三个独立模块并整合进 `_shared/` 供其他两人调用。

BM25 中文索引最初用的 `char_terms` 分词（每两个字切一块），效果很差——“游泳池”切成“游泳”和“泳池”，“早餐”处理成一个字。换成 `jieba` 分词后查准率直接拉到接近满分。参数用了标准 BM25 ($k_1=1.5, b=0.75$)，没有调参，在这个数据集上标准参数已经够好。索引规模：2171 篇文档，2504 个词项，构建 0.1s，单次查询平均 2.64ms。核心类 `InvertedIndex` 约 120 行。

查询词	Top Score	说明
早餐	4.04	高频词，区分度可
游泳池	12.67	低频词，精确匹配
服务	1.56	抽象概念，分偏低
噪音	11.92	低频精确词
停车场	12.80	匹配精准

意图分类最早用的是英文关键词分类 (`qa / visual / mixed`, 3 类)，准确率只有 **7.1%**，基本没用。改成中文 10 维后准确率到 **96.4%**。28 条测试中唯一失败的 case：值得推荐吗 → `price`，因为“值”被当成了 `price` 类关键词“值得”。代码在 `_shared/router.py` 的 `classify_intent_detailed()`。

聚类分析方面，用 `TF-IDF + UMAP + HDBSCAN` 做了无监督聚类（代码在 `scripts/dyx/category/`），目标是数据驱动发现评论主题，与人工标注对比验证。文本向量化用 `TF-IDF` (`max_features=3000`)，降维用 `UMAP` 两阶段（先 50 维供聚类，再 2 维供可视化），聚类用 `HDBSCAN` (`min_cluster_size=30`) 无需预设簇数。最终发现 14 个聚类，与人工标注类别数巧合一致。

施工噪音簇（簇 8）曾误匹配餐饮内容，因为 `TF-IDF` 均值中心包含“早餐”等跨簇词汇。引入 `c-TF-IDF`（簇间 `IDF` 加权）后，簇 8 判别词从“早餐、自助”变为“施工、噪音、太吵”，问题解决。`HDBSCAN` 未产生独立餐饮簇，补充了手工餐饮伪簇覆盖 236 条评论（10.9%）。

与人工标签对比：聚类优势在粒度更细（行政酒廊 152 条独立于普通餐饮）、发现了人工未设定类别（施工噪音 62 条、节日氛围 67 条）、提取了行为信号（回头意向 192 条）。不足在于平均仅 1.49 标签/条（人工 2.80）、44.1% 噪声点、短评效果差。最终在问答系统中仍采用人工标签，因为聚类标签的业务语义清晰度不如人工分类，且覆盖率偏低。聚类分析作为探索性研究保留。

类别摘要检索是 `RAG` 问答系统的第五路检索，目标是在回答用户问题时提供宏观类别背景，帮助 `LLM` 做出更全面的判断。知识库包含 14 个业务类别摘要（前台服务、餐饮设施、房间设施等），每条约 650 字。摘要生成采用分层聚合策略：先分批调用 `qwen-turbo` 生成批次小结和关键词，再词频统计整合 `top-5` 关键词，最后合成最终摘要。

检索采用非对称检索：离线用 `embed_batch()` 对“类别名：关键词。摘要前 300 字”向量化，在线用 `embed_query()` 传入任务指令激活 `text-embedding-v4` 的指令微调能力。三层优化：相似度阈值过滤 ($\tau=0.25$)，意图规则混合召回（向量检索未命中但意图关键词命中时强制补充），`QA` 变体生成（每类别额外 3 条假设句作为检索桥接层）。召回的摘要以“类别背景知识”块注入 `prompt`，形成“宏观背景 + 具体证据”的双层结构。代码在 `scripts/dyx/engine.py`。

三、场景顾问 Agent

代码在 `scripts/advisor/agent.py` (主控)、`recognizer.py` (场景识别)、`scenes.py` (场景-维度矩阵)、`synthesizer.py` (综合研判)。这是第三阶段新做的,把三套算法从“搜索引擎”改造成了“顾问”。

实际测试中用户输入天然分成两类。“带 5 岁宝宝去住推荐吗”是顾问咨询型,用户不知道酒店对亲子家庭来说怎么样,需要多维度综合分析。“有洗衣机吗”是事实查证型,只需要 yes/no 加证据。把两种混在一起处理的问题很明显——事实查证型走维度展开会搜出一堆不相关的东西(“有洗衣机吗”被拆成“退房效率+入住时间+前台服务”三个维度),而顾问咨询型走直接检索又给不出结构化的多维度评估。

第一版只做了 6 个出行场景(商务/亲子/蜜月/长辈/朋友/独自),靠 LLM 分类。测出来的第一个大 bug:“半夜 12:30 到达可以入住吗”被 LLM 分到了“独自旅行”。LLM 看到了“我”字,倾向 solo,然后拆维度搜了“位置交通+安全+性价比”,跟用户想问的“凌晨能不能办入住”完全不搭。这个 case 暴露了 6 场景体系的根本缺陷:用户问酒店实务问题(入住时间、外卖快递、宠物政策、客房设施)的时候,不属于任何一个“出行场景”。于是加了第七个场景 **practical** (实务咨询),并设计了关键词预检机制。

关键词优先于 LLM,纯粹是工程考虑。正则匹配微秒级,不调 API 不花钱,而且高频短语命中率高——用户翻来覆去问的就是凌晨入住、延时退房、外卖快递、枕头材质、宠物政策这些事。实测关键词截住约 80% 的实务查询。LLM 用来兜底,关键词总有漏的。“帮订景区门票”、“床垫偏硬还是偏软”正则罩不住,LLM prompt 里扩展了 practical 场景定义(包含 10+ 个具体例子)来覆盖长尾。

关键词的覆盖是打补丁打出来的。最初只有 7 类(入住、退房、行李、接送、支付、设施、宠物),每测出一个新盲区就补。这个迭代持续了好几轮:“可以带猴子入住吗”暴露了“宠”字正则不够宽,补了“带.*动物”;“能借 HDMI 线吗”没命中,因为 query 被 lower() 了小写而 HDMI 是大写,补了 `re.IGNORECASE`;“能在房间抽烟吗”命中了 `special_req`,但维度 key 叫 `special_req` 而 subtype 叫 `special_request`,名字不一致导致维度没选对,统一了命名;“提供免费避孕套吗”LLM 把“免费”加“套”理解成了蜜月场景,补了 `room_amenity` 模式。最终关键词体系稳定在 14 类、100+ 条正则,代码在 `recognizer.py` 的 `_PRACTICAL_PATTERNS` 字典里。

practical 的管线也改过三版。第一个版本 **practical** 也走维度展开,“有洗衣机吗”拆成退房效率+入住时间+前台服务三个维度各自检索,搜出一堆跟洗衣机无关的退房和入住评论。第二版只查核心维度加匹配 subtype 的次要维度,好了一点,但核心维度(退房、入住、前台)对洗衣机查询还是不相关。第三版,也就是现在用的版本, **practical** 不走维度展开,直接用原始问题的关键词去搜,6 个事件完成(advisory 要 16 个),延迟从 5s 降到 0.3s。去重、核心词过滤、展示,搜不到就诚实说没找到。

路由准确率:关键词 pre-check 100% (15 条高频),LLM extended prompt 100% (4 条边缘),综合路由 100% 非 general (23 条),Advisor 全量 100% (149 条)。

管线延迟: **practical** 关键词路径 6 事件 0.3s 零 API 调用; **practical** LLM 路径 6 事件 1.1s 一次 LLM 调用; advisory LLM 路径 16 事件 5.4s 两次 LLM 调用。

四、测试体系

测试是这个项目第三阶段投入最大的部分之一。第一、第二阶段几乎没有测试——三个人各自开发，改代码全靠手工验证。到整合的时候，改一个路由逻辑可能同时影响三个人的模块，一跑就崩。于是第三阶段花了大量精力建测试体系，现在全部 **980 条测试**，一键 `python tests/run_all.py`。

测试设计遵循几个原则。每个算法模块有独立的召回指标基准，不能因为改了路由就让检索精度退化。每个路由决策点有对应的正确性测试，改场景分类必须跑过所有已有 case。性能关键路径（BM25 查询延迟、意图分类延迟）有基准线，不能接受性能回退。所有测试结果写 JSON 存档到 `tests/results/`，方便追溯。

引擎单元测试（30 条）

代码在 `tests/unit/test_all.py`。这是最早建的一组测试，覆盖 Layer 1 三个人的核心算法，确保整合时各自的模块没有悄悄坏掉。

意图分类 16 条，覆盖 10 个中文维度各 2-3 条测试用例。每条测试有明确的 `expected_intent`，从“游泳池干净吗 → `facility`”到“适合带孩子住吗 → `child_friendly`”，覆盖高频查询和边界情况（空查询、纯英文、特殊字符）。董依心的 `classify_intent_detailed` 在这 16 条上做到 96.4%，唯一失败 case 是“值得推荐吗”的“值”字误匹配 `price` 类。

BM25 搜索 4 条，每条验证 top-10 结果非空且延迟在合理范围内。RAG 管线 3 条，验证 `evidence_score > 0` 且 `categories` 非空。RRF 融合 2 条，验证双路结果能正确融合。路由集成 3 条，验证 `retrieve()` 和 `route()` 两个统一入口在热缓存下正常返回。性能基准 2 条：BM25 构建 2000 篇文档 < 2s，10 次查询平均 < 5ms。30/30 通过。

Advisor 模块测试（149 条）

代码在 `tests/unit/test_advisor.py`。这是第三阶段新建的，也是最大的一组测试。它的构建方式和前一组完全不同——不是在开发完成后补的，而是在改路由逻辑的过程中，每发现一个路由错误就加一条测试。这个 workflow 变成了：用户报告“半夜入住被分到 solo” → 写一条测试复现 → 修 `_detect_practical` → 跑测试确认 → 加一条回归测试防止再犯。整个过程下来，Advisor 测试从 0 条涨到了 149 条。

这 149 条分成四个模块，和 Advisor 的四个源文件一一对应。

Recognizer 测试 38 条，测 `_detect_practical` 和 `recognize` 两个函数。关键词预检部分覆盖了全部 14 个实务类别（`checkin`、`checkout`、`luggage`、`shuttle`、`payment`、`room_amenity`、`dietary`、`health_safety`、`tech_service`、`service_misc`、`pet`、`special_req`、`accessibility`），每个类别至少 2 条测试。特别设计了一组“离谱但该命中”的查询——“可以带猴子入住吗”（`pet`）、“酒店提供免费避孕套吗”（`room_amenity`）、“能在房间烧香拜佛吗”（`special_req`）、“我要在酒店里开 30 人的 party”（`special_req`）——确保正则覆盖足够宽。另外有一组“不该命中”的查询——“带 5 岁宝宝去住”（亲子场景，不应被 `practical` 拦截）、“下周出差开会”（商务）——确保关键词不会误吞正常场景查询。`keyword_fallback` 测试覆盖了 6 个出行场景的关键词回退逻辑，从“宝宝 → `family`”到“一个人 → `solo`”，再到无意义查询 → `general`。

Scenes 测试 70 条，测场景-维度矩阵的声明式数据完整性。逐个验证 7 个场景（business、family、romance、elder、friends、solo、practical）在 SCENE_LABELS 中都有中文标签；每个场景的核心维度数 ≥ 1 ；每个核心维度在 QUERY_TEMPLATES 中有非空 query 模板；DIMENSION_INFO 字典包含所有维度的 label 信息。这组测试看起来简单，但它保证了一件事：新增场景或调整维度时，如果忘了同步更新三个字典（MATRIX、DIMENSION_INFO、QUERY_TEMPLATES），测试直接报错。

Pipeline 测试 41 条，分 advise 和 advise_stream 两部分。advise 部分 19 条，测试端到端管线的路由正确性——practical 查询走直搜路径（dims 为空）、advisory 查询走维度展开、回复非空、边缘查询不崩溃。advise_stream 部分 22 条，测试流式输出的事件序列完整性——第一个事件总是 recognizing、最后一个总是 done、中间的 phase 都在合法集合内、done 事件包含 reply、所有 msg 不含 emoji。这组测试在把 practical 从“维度展开”改成“直接检索”的时候救了命——改完一跑，practical 的 dims 预期要空，结果老代码还在展开，测试直接红了。

149/149 通过。测试覆盖了 14 类实务关键词，外加 23 条故意刁难的边缘查询。

集成测试（801 条）

路由器评测（tests/integration/test_router.py）28 条。测试 route() 完整调用链，含意图分类和策略编排。每条 query 有 expected_intent，覆盖全部 10 个中文维度。还做了延迟评测：意图分类中位数 9.6 μ s，route() 热缓存下平均 14ms。通过率 96.4%，唯一失败同单元测试。

多酒店扩展测试（tests/integration/test_extended.py）48 条。从 data/hotel_reviews_full.parquet（694 家酒店）中抽 12 家各 200 条评论，构建跨酒店 BM25 索引。测试意图分类在跨酒店数据上的泛化能力、BM25 跨酒店检索是否有结果、各酒店自己的评论能被搜到。通过率 89%，失败集中在短评和跨酒店歧义。

大规模多酒店测试（tests/integration/test_multi_hotel.py）725 条。全量测试意图分类 + BM25 在所有 12 家酒店上的表现，包含边界条件（空查询、超长查询、纯数字）。通过率 92%。这组测试跑一次大约 3 分钟，通常只在改 BM25 索引或分词逻辑时跑。

召回基准

10 条标准查询，text-based keyword relevance，横评四种算法。P@K 定义为 top-K 结果中与查询关键词匹配的比例。这个基准的设计意图不是“证明哪个算法最好”，而是**建立基线，防止回退**——如果有人改了 BM25 的分词逻辑，要能立刻看到 P@5 从 100 掉到 95 了。

Metric	BM25	RAG	Fusion	CLIP
P@1	100	100	100	50
P@5	100	96	100	36
MRR	100	100	100	60
延迟	2.6ms	~500ms	5-10ms	~500ms

CLIP 的 P@1 只有 50 不是因为它不行——它是图文检索引擎，用文本关键词来评价图片检索本身就存在偏差。查”停车场”返回的是拍了停车场的照片，但关联评论不一定写”停车场”这个词。图片级别的 ground truth 没建，这个指标仅供参考。

五、分工

成员	核心贡献	主要文件
苏俊儒	Chinese-CLIP 多模态检索；自研 NumPy 向量存储（替代 ChromaDB）；LTR 7 轮迭代（发现天花板在召回）；6 组微调实验（均失败）；BERT 房型分类器；OpenVINO GPU 加速（23-34×）；召回评测	scripts/leuca/multimodal/、scripts/_shared/store.py
孙宇豪	Agentic RAG 流水线（Plan→Retrieve→Evaluate→Rewrite）；SFT 1200 条数据 + QLoRA 微调（未部署）；LLM 统一接口（_shared/llm.py）；双层 prompt 设计	scripts/yuhao/agentic/、scripts/yuhao/sft/、scripts/_shared/llm.py
董依心	BM25 中文倒排索引（jieba, 2.64ms）；10 维中文意图分类（96.4%）；RRF 融合；QA 变体桥接层；HyDE 实验；聚类分析	scripts/_shared/bm25.py、router.py、fusion.py、scripts/dyx/
整合层	全局缓存（_shared/cache.py）；BM25-first 策略（_shared/retriever.py）；场景顾问 Agent（scripts/advisor/）；CLI（hotel/）；测试体系（980 条）	scripts/_shared/、scripts/advisor/、hotel/

六、反思

大部分酒店查询就是精确关键词匹配。BM25-first 策略在实际使用中覆盖约 70% 的查询（_shared/retriever.py:144-166），这是从使用数据中反推出来的优化，不是设计阶段想到的。中文 10 维意图分类微秒级、96.4%、零成本，比改了一堆 prompt 的 LLM 路由可靠得多——酒店评论查询的意图空间本来就有上限，用户来来回回就是问早餐、游泳池、位置、服务这些事。

关键词加 LLM 两级路由是反复踩坑后收敛到的最优解。纯关键词从 7 类改到 14 类，每次测出盲区就往里补，这条路有天花板。纯 LLM 又贵又不稳定（“半夜入住”被分到

solo)。各取所长刚好互补：关键词管高频且免费，LLM 兜底泛化覆盖长尾。Advisor 的 149 条测试是在改路由逻辑过程中边踩坑边加的。有了测试之后改路由策略不再心慌，改完跑一遍全绿就放心。这个习惯如果从项目一开始就建立，整合阶段能省一半时间。

CLIP 多模态的天花板是选择 Chinese-CLIP 这个预训练模型时就决定的上限。5.4% Hit@10，微调全部失败，LTR 撞 7%。同一评论的文本-图片余弦相似度只有 0.36，CLIP 的预训练空间里根本没有对齐好酒店评论文本和酒店实拍图片。SFT 数据做了、权重产出了，卡在 QLoRA 和 OpenVINO 的兼容性上——如果一开始用 FP16 全量微调而不是 4-bit QLoRA，兼容性问题可能不存在，但 Arc GPU 的显存不一定够。意图分类器有三套关键词映射分布在 `_shared/router.py`、RAG 管线和 advisor 的 `_detect_practical` 里，改了一个另一个不同步，后来在 advisor 统一了但历史包袱还在。“双人房能住三人吗”这种政策问题在评论语料里天然没有答案，系统能做的就是搜不到的时候诚实说没找到。

如果再做一遍：先定接口协议再各自实现，整合阶段花的时间比预计翻了一倍，如果一开始就约定 `retrieve(query, top_k, strategies)` 这个统一入口后面不用来回改。BM25 够用就别上复杂模型，CLIP 在这个数据集上的回报跟复杂度完全不成正比。测试早点写，第二阶段的测试空白是第三阶段整合时最痛苦的地方。LTR 从 R2 到 R6 如果早点分析 CLIP 召回池的质量数据（top-20 仅 1.4% 命中），就不会浪费 4 轮迭代在撞墙。两级路由值得一开始就设计进去，收敛过程很漂亮但收敛过程太痛苦了。

报告完。